Folio: AI-Assisted Mathematical Workspace

COMPSCI 4ZP6A/B
Group 10
SRS & Project Plan Document V0.1 Baseline
October 9th, 2026
Instructor: Dr. Moradi (moradm4@mcmaster.ca)

Part I: Front Matter & Governance 2
Personnel & Roles 2
Definitions, Acronyms & Abbreviation 2
Contributions Table 3
Executive Summary & Purpose 4
Part II: Software Requirements Specification (SRS) 5
Stakeholders & User Personas 5
Constraints & Assumptions 6
Functional Requirements 6
Data Strategy & Quantitative Metrics 10
Non-Functional Requirements 11
Risk Management & Feasibility Analysis 12
Part III: Project Development & Execution Plan 13
Team Communication & Meeting Cadence 13
Engineering Workflow & Version Control 14
Infrastructure & Compute Resources 14
Technology Stack & Architectural Rationale 15
Proof of Concept (PoC) Demonstration Plan 16
Project Schedule & Gantt Chart 16

Part I: Front Matter & Governance

Personnel & Roles

| Full Name | McMaster Email Address | Student Number | Role |
|---|---|---|---|
| Dev Mody | modyd1@mcmaster.ca | 400445340 | Project Manager, ML/Recognition Pipeline, Frontend Engineering (iPad) |
| Meet Patel | patem196@mcmaster.ca | 400455675 | ML/Recognition Pipeline, Data Processing |
| Jasimraza Momin | mominj@mcmaster.ca | 400462671 | iPad Full-Stack, ML/Recognition Pipeline |
| Aryaan Sheth | shetha3@mcmaster.ca | 400377764 | Desktop Full-Stack, Infrastructure |
| Saad Khalid | khalid68@mcmaster.ca | 400449682 | Desktop Full-Stack, ML/Recognition, Agent Development |

Definitions, Acronyms & Abbreviation

| Term / Acronym | Definition |
|---|---|
| Math IR | Mathematical Internal Representation: Folio's structured data format for representing mathematical expressions as editable, maniputable objects |
| HMER | Handwritten Mathematical Expression Recognition |
| CROHME | Competition on Recognition of Online Handwritten Mathematical Expressions (ICDAR benchmark dataset) |
| MathWriting | Google Research dataset of online handwritten mathematical expressions in InkML format |
| HME100K | Large-scale handwritten math expression dataset with diverse writer style |

| Term / Acronym | Definition |
|---|---|
| InkML | XML-based format for representing digital ink (handwriting stroke data) |
| SymLG | Symbol-Level Label Graph: structural ground truth used in CROHME for symbol and spatial relationship annotation |
| ExpRate | Exact Expression Rate: percentage of model predictions matching ground truth exactly |
| CDM | Character Detection Matching: evaluation method for visually equivalent LaTeX expressions (used by UniMERNet) |
| VLM | Vision-Language Model: a neural network trained on both visual and textual data |
| UniMERNet | Specialized neural network architecture for mathematical expression recognition |
| Uni-MuMER | Fine-tuned QWEN vision-language model for handwritten math recognition |
| P0–P3 | Priority tiers: P0 (must-have MVP), P1 (high priority), P2 (medium), P3 (stretch) |
| RREF | Reduced Row Echelon Form |
| WCAG | Web Content Accessibility Guidelines |
| PencilKit | Apple framework for capturing and rendering Apple Pencil stroke data on iPadOS |
| SymPy | Python library for symbolic mathematics used for deterministic command computation |

Contributions Table:

| Team Member | Sections Authored | Sections Reviewed | Estimated Contributions (%) |
|---|---|---|---|
| Dev Mody | 9, 13, 2, 3, 4, 8 | All | 20% |
| Jasimraza Momin | 9, 13, 2, 3, 4, 8 | All | 20% |
| Meet Patel | 11, 14, 15, 2, 3, 4, 8 | All | 20% |

| Team Member | Sections Authored | Sections Reviewed | Estimated Contributions (%) |
|---|---|---|---|
| Saad Khalid | 7, 10, 12, 16, 17, 2, 3, 4, 8 | All | 20% |
| Aryaan Sheth | 1, 5, 6, 2, 3, 4, 8 | All | 20% |

Executive Summary & Purpose:

Folio is a mathematics workspace for macOS and iPad that lets users solve problems and produce submission-ready PDFs in one workflow. Students solve math faster by hand than by typing, but submitting typed solutions means rewriting everything in LaTeX. Folio removes that rewrite.

On macOS, users enter math through keyboard input with context-aware shortcuts inspired by Vim and Emacs. The shortcuts change based on the current task, selected work, and subject context. AI-powered shortcuts turn intended operations into mathematical notation, so users type less and work faster. On the iPad, users write with the Apple Pencil. The app converts handwriting into structured, editable typeset math. Users correct recognition errors before accepting results. Both platforms share a common document format, so work transfers between devices.

The core technical contribution is a pipeline that captures handwritten or typed expressions, converts them into a structured internal representation (Math IR), and runs symbolic manipulation through commands: `/rref`, `/matmul`, `/derivative`, `/integral`, `/simplify`. Recognition and computation are separate. A verified structured expression can be manipulated with deterministic correctness even if handwriting recognition made mistakes upstream. On macOS, where input is structured, commands are high priority. On iPad, commands follow once the recognition pipeline is reliable.

The first release targets linear algebra, calculus, and proofs. Features: mathematical input and editing, shortcuts for duplicating and modifying previous steps, matrix operations, selected mathematical transformations, AI-assisted step checking, local persistence, and PDF export. AI helps users check steps, catch errors, and convert intended operations into proper notation. Both apps preserve step order and explanations when saving and exporting. Editing shortcuts work on unfinished expressions so users keep solving without stopping to fix formatting.

Testing compares Folio against handwriting alone and handwriting followed by LaTeX transcription. Metrics: completion time, input errors, export correctness, usability after a set practice period, and submission readiness.

## Part II: Software Requirements Specification (SRS)

### Stakeholders & User Personas

Primary Stakeholders: Due to the nature of Folio as a student developed project, the team acts as the initial client, using our collective experience with handwritten math and LaTeX transcription to define the core requirements needed for a smooth workflow. Feedback from other students outside the team will help check whether these requirements address a wider need and if any additional features are needed.

Secondary Stakeholders:

| Stakeholder | Project Interest |
|---|---|
| Instructors & TA's | Readable properly formatted submissions that show proper mathematical steps and explanations |
| Capstone Supervisor | Appropriate score, technical depth, and measurable results |
| Development Team | Maintainable code and feasible and clear development plan |
| Student Participants | An efficient interface that allows proper handling of work and feedback on math |

User Personas: These archetypes describe intended users and will be validated through user testing.

| User Type | Current Problem | Needs |
|---|---|---|
| Keyboard-focused student | Typing math takes longer than handwriting, followed by additional time rewriting solutions in LaTeX for submissions. | Fast input, context-aware shortcuts, reuse of previous steps, and PDF & Raw LaTeX export. |
| Handwriting-focused student | Prefers paper or iPad but must retype work for a formatted submission. | Handwriting tools, conversion to typeset math, recognition corrections, and PDF export. |

| | | |
|---|---|---|
| Student completing long calculations or proofs | Repeatedly copies expressions, changes individual terms, and adds justifications. | Step duplication, targeted editing, matrix tools, undo/redo, and explanations alongside math. |

Constraints & Assumptions

The software will be designed for modern web browsers, including Chromium, Microsoft Edge, Firefox, and Opera, as well as native macOS and iPadOS applications. Operating under generally accepted consumer privacy and safety guidelines, the project will leverage open-source libraries and software with appropriate attribution whenever possible, with no reliance on proprietary third-party software. Additionally, it is assumed that user hardware across all defined personas will meet the baseline performance capabilities required to run the application efficiently.

Functional Requirements

| ID | Priority | Platform | Requirement | Acceptance check |
|---|---|---|---|---|
| FR-01 | P0 | Both | Create, save, and reopen local documents | Reopened documents preserve content, ordering, and handwriting strokes on iPad |
| FR-02 | P0 | macOS | Enter and edit mathematical expressions (fractions, powers, subscripts, roots, integrals, matrices) using keyboard input and shortcuts | Every expression in the agreed input benchmark can be entered and edited |
| FR-03 | P0 | iPad | Write and draw mathematical work using pen input, with selection, movement, and erasure tools | Users can modify selected strokes without affecting unrelated content |
| FR-04 | P0 | iPad | Convert selected handwritten mathematics into typeset expressions, | Benchmark expressions are recognized or corrected, then inserted without deleting original work prematurely |

| | | | | |
|---|---|---|---|---|
| | | | with correction before acceptance | |
| FR-05 | P0 | Both | Display typeset mathematical expressions alongside working content | Supported expressions render correctly; incomplete or invalid input remains editable |
| FR-06 | P0 | Both | Select and capture a mathematical row or region as a discrete object for later processing | Selected region becomes a bounded input object usable by downstream recognition and commands |
| FR-07 | P0 | Both | Duplicate previous mathematical steps and edit copies independently | Editing a duplicated expression or stroke group leaves the original unchanged |
| FR-08 | P0 | Both | Combine mathematical steps with explanatory text in sequence | Saving and export preserve intended sequence of expressions and explanations |
| FR-09 | P0 | Both | Undo and redo document edits | Supported edits can be reversed and reapplied without losing unrelated work |
| FR-10 | P0 | Both | Export a typeset PDF from the document without separately rewriting the solution in LaTeX | Benchmark documents export without missing expressions, clipped content, or incorrect ordering |
| FR-11 | P1 | Both | Represent supported mathematical expressions in a structured internal representation (Math IR) that enables programmatic manipulation | ≥95% of successfully recognized supported inputs produce syntactically valid Math IR |

| | | | | |
|---|---|---|---|---|
| FR-12 | P1 | macOS | Execute /rref on a valid Math IR matrix and insert the resulting reduced-row-echelon form as a new editable row | Result matches known answers; ≥99% correctness on deterministic test cases |
| FR-13 | P1 | macOS | Execute /matmul on two compatible Math IR matrices and return their product | Result matches trusted symbolic implementation; ≥99% correctness |
| FR-14 | P1 | macOS | Execute /derivative on a supported differentiable Math IR expression and return a symbolically equivalent derivative | Result matches SymPy output; ≥99% correctness |
| FR-15 | P1 | macOS | Execute /integral and /simplify on supported Math IR expressions | Results match symbolic math library output; ≥99% correctness |
| FR-16 | P1 | macOS | Apply supported matrix row operations to a selected matrix | Results match known answers and include the operation performed |
| FR-17 | P1 | macOS | Substitute a selected variable or expression within a supported mathematical expression | Substitutions match known-answer cases and preserve the original step |
| FR-18 | P1 | Both | Choose which steps and scratch work appear in the exported document | Excluded content remains saved but does not appear in PDF |
| FR-19 | P1 | Both | Request AI checks, hints, or explanations for selected work | Feedback uses selected context; proposed changes require user acceptance |

| | | | | |
|---|---|---|---|---|
| FR-20 | P2 | iPad | Execute /rref, /matmul, /derivative, /integral, and /simplify commands on valid Math IR expressions recognized from handwriting | Same correctness targets as macOS commands; gated behind reliable iPad Math IR |
| FR-21 | P2 | Both | Preserve and navigate alternative solution branches | Users can revisit either branch without losing its steps |
| FR-22 | P2 | Both | Transfer documents between macOS and iPad using a shared file format | Supported content survives transfer and reopening on either platform |
| FR-23 | P2 | Both | Generate linked 2D visualizations from supported expressions | Visualizations reflect the selected expression and update when it changes |
| FR-24 | P2 | macOS | Create custom user-defined commands that chain built-in operations or define reusable mathematical transformations | Custom command executes correctly on supported Math IR inputs and produces editable results consistent with its definition |
| FR-25 | P3 | Both | Extend structured recognition to broader complex handwritten mathematical expressions (nested structures, matrices, multi-line derivations, integrals, summations, limits) | Higher ExpRate on CROHME and full MathWriting test sets with minimal manual correction |

Data Strategy & Quantitative Metrics

Folio uses public handwriting datasets, synthetic data, and internal benchmarks to develop and evaluate its mathematical recognition and reasoning systems.

Data Sources:
The primary datasets include:

- MathWriting: Main handwriting dataset with over 230,000 real and 396,000 synthetic mathematical expressions.
- CROHME 2023: Handwritten expressions with LaTeX and structural annotations for independent evaluation.
- HME100K: Optional dataset for greater handwriting variation.
- Internal Benchmark: Team-generated expressions covering algebra, calculus, matrices, fractions, and varying handwriting quality.

Synthetic expressions will supplement public datasets, while training and evaluation data remain separate to ensure reliable comparisons.

Recognition and AI Models: Folio will compare specialized recognition models (UniMERNet), fine-tuned vision-language models (Uni-MuMER/Qwen), general multimodal models (Gemini), and a custom structure-first pipeline that converts handwriting into Math IR. Models will be evaluated based on accuracy, structural correctness, latency, computational requirements, and deployment costs.

Folio also plans to post-train open-source language models (8B-70B parameters) using Modal or AWS for mathematical reasoning, step verification, hints, and explanations. Training datasets include PRM800K, MATH, NuminaMath-CoT, OpenMathInstruct-2, and internally generated data. A proposed reinforcement learning environment will support mathematical proof verification across algebra, calculus, linear algebra, discrete mathematics, combinatorics, probability, and statistics. These advanced AI features will remain separate from Folio's core recognition and Math IR functionality.

Evaluation Metrics:
System performance will be measured using:

- Recognition Accuracy: Exact expression matching (ExpRate) and canonical equivalence (CDM).
- Structural Accuracy: Correct representation and conversion of expressions into Math IR.
- Capture Success: Accurate identification of handwritten regions.
- Performance: Recognition latency, command execution time, and computational cost.

Quantitative Targets:

| Component | Target |
|---|---|
| P0 – Row Capture | ≥95% success |
| P1 – Simple Expression Recognition | ≥90% structural accuracy |
| P1 – Math IR Conversion | ≥95% valid conversion for correctly recognized inputs |
| P1/P2 – Mathematical Commands | ≥99% correctness |
| End-to-End Recognition | ≥85% success |
| Command Execution | <1 second |
| Recognition Latency | <2 seconds |
| CI Validation | 100% required checks passing |

These targets will be evaluated using fixed test sets, with public benchmark results reported separately to track model performance and improvements.

Non-Functional Requirements

| ID | Priority | Description |
|---|---|---|
| NFR-01 | P0 | The application should start up in less than 1 second, and all navigation between pages and screens should occur within 250 ms. |
| NFR-02 | P0 | All large language model latencies for non-chat interfaces should be under 300 ms. |
| NFR-03 | P0 | 95% of large language model responses should be relevant to the context and the requested action. |

| | | |
|---|---|---|
| NFR-04 | P0 | All mathematical data remains in a representation that can be losslessly converted into LaTeX |
| NFR-05 | P1 | Recovery: After an unexpected application close, Folio shall recover the most recent automatically saved document state, losing no more than 10 seconds of work. |
| NFR-06 | P1 | Export reliability: At least 95% of documents in the agreed export test set shall produce a PDF without missing content, clipped mathematics, or incorrect ordering. |
| NFR-07 | P1 | AI failure handling: If an AI request fails or times out, Folio shall preserve the user's work and show a recoverable error within 2 seconds after the failure is detected. |
| NFR-08 | P1 | Accessibility: All core keyboard workflows shall be operable without a mouse, and the team shall evaluate the interface against WCAG 2.2 AA criteria applicable to the supported platform. |

Risk Management & Feasibility Analysis
Folio is feasible for a five-person team over two semesters:

- A single TypeScript codebase serves the web, macOS and iPad apps.
- Matrix operations and substitution are computed exactly by our own engine, never by a language model.
- Every handwriting recognition result and every AI suggestion has to be accepted by the user before it changes their work.

The main risks are handwriting recognition accuracy, AI response speed (NFR-02), and GPU availability. All three will be tested before the November 22 proof of concept.

| ID | Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|---|
| R-01 | Handwriting recognition is not accurate enough | Medium | High | Build the correction screen first (FR-04). Limit recognition to the symbols the P0 features need. Measure accuracy on our test expressions before the proof of |

| | | | | |
|---|---|---|---|---|
| | | | | concept. |
| R-02 | AI responses are slower than the 300 ms target (NFR-02) | Medium | Medium | Use the 8B model for quick in-editor actions and keep larger models for chat. Stream responses and cache repeated requests. Measure speed every sprint. |
| R-03 | CAS GPUs are unavailable or not powerful enough | Medium | Medium | Request access and confirm specs with CAS in October. Move more training and serving to Modal and AWS if needed, with spending caps and alerts. |
| R-04 | Supporting web, macOS and iPad overloads the team, or Apple Pencil input lags in the iPad web view | Medium | Medium | Keep all logic in the shared core. Only P0 features need to work identically on every platform. Build a pen prototype in the first sprint; if it is too slow, move the iPad app to native Swift. |
| R-05 | Team availability drops during exam periods | Medium | Medium | Plan lighter sprints around exams. Have at least two people working on each critical component. |

# Part III: Project Development & Execution Plan

## Team Communication & Meeting Cadence

We will be meeting 2x weekly on Saturdays and Thursdays to discuss engineering, product and vision. Other meetings are done on an ad-hoc basis but we will have these two meetings every week for all hands. Our meeting cadence is documented in our github repository

## Engineering Workflow & Version Control

Folio uses GitHub for version control and pull-request reviews, and Linear for issue tracking and sprint planning. A monorepo structure keeps the frontend, backend, mathematical processing, machine learning, tests, and documentation together to simplify collaboration and integration.

Repository Structure:

folio/
- ↳src/
  - ↳ frontend/ – SwiftUI interface (iPadOS), React Electron (Desktop)
  - ↳ backend/ – APIs, persistence, and orchestration
  - ↳ math/ – Math IR, parsing, commands, and equivalence
  - ↳ ml/ – Preprocessing, inference, and evaluation
- ↳tests/ – Unit, integration, and end-to-end tests
- ↳benchmarks/ – Fixed evaluation benchmarks
- ↳datasets/ – Dataset manifests and processing scripts
- ↳docs/ – SRS and technical documentation
- ↳meetings/ – Team meeting records
- ↳scripts/ – Build and utility scripts
- ↳.github/workflows/ – CI configuration

Development occurs on short-lived branches linked to Linear issues, with main kept stable. Pull requests require passing automated checks, relevant tests, and at least one team member's approval. GitHub Actions handles formatting, static analysis, builds, and automated testing, while larger ML evaluations run separately.

The team follows approximately two-week sprints, prioritizing tasks with clear acceptance criteria and reviewing progress and blockers at each cycle's end. Tasks are complete once acceptance criteria are met, tests pass, documentation is updated, and

changes are merged. ML tasks additionally require recorded configurations and evaluation results.

Infrastructure & Compute Resources

- Team Macs and at least two iPads with Apple Pencil. iPad builds and Apple signing require a Mac.
- CAS GPUs, Modal and AWS are used to post-train open-source models from 8B to 70B parameters on public and synthetic data.
- Personal 4060Ti (8GB) and 5070Ti (16GB)
- The post-trained models run on Modal or AWS. Cloud servers shut down automatically when idle and have spending caps.
- A cloud virtual server hosts the API. It handles requests and rate limiting and connects the apps to the model servers. It does not store user documents.
- GitHub Actions runs the tests.
- The web app is hosted on vercel. The iPad app is delivered through TestFlight. The macOS and iPad apps require an Apple Developer Program membership (USD $99 per year).

Technology Stack & Architectural Rationale

| Layer | Choice | Rationale |
|---|---|---|
| Frontend | TypeScript and React | One shared codebase for the web, macOS and iPad apps. |
| Desktop (macOS) | Electron* | Full keyboard control, local file access and PDF export. |
| Tablet (iPad) | Capacitor* | Runs the same web app natively on iPadOS. |
| Backend | Python and FastAPI | A lightweight API that connects the apps to the model servers. |
| ML | Open-source LLMs, PyTorch | Post-training on CAS GPUs, Modal and AWS. |
| Testing | Vitest, Playwright, pytest | Unit tests for core logic, end-to-end tests for user workflows, and backend |

| | | |
|---|---|---|
| | | tests. |

*Electron with Capacitor is the first attempt. If platform-specific problems come up, we may move that platform to native Swift.

Proof of Concept (PoC) Demonstration Plan

| Target Date | Milestone / Deliverable |
|---|---|
| Oct 18 | Editable notebook; save and reopen |
| Oct 25 | Duplicate steps and undo |
| Nov 1 | Check a selected step |
| Nov 8 | Step-linked, hint-first feedback |
| Nov 15 | Stale-result handling and uncertainty labels |
| Nov 22 | Complete, demo-ready workflow and video |

Project Schedule & Gantt Chart

The schedule below presents the planned work from October 2026 through the April 2027 final submission. Dates are estimates; course milestones are shown in green.

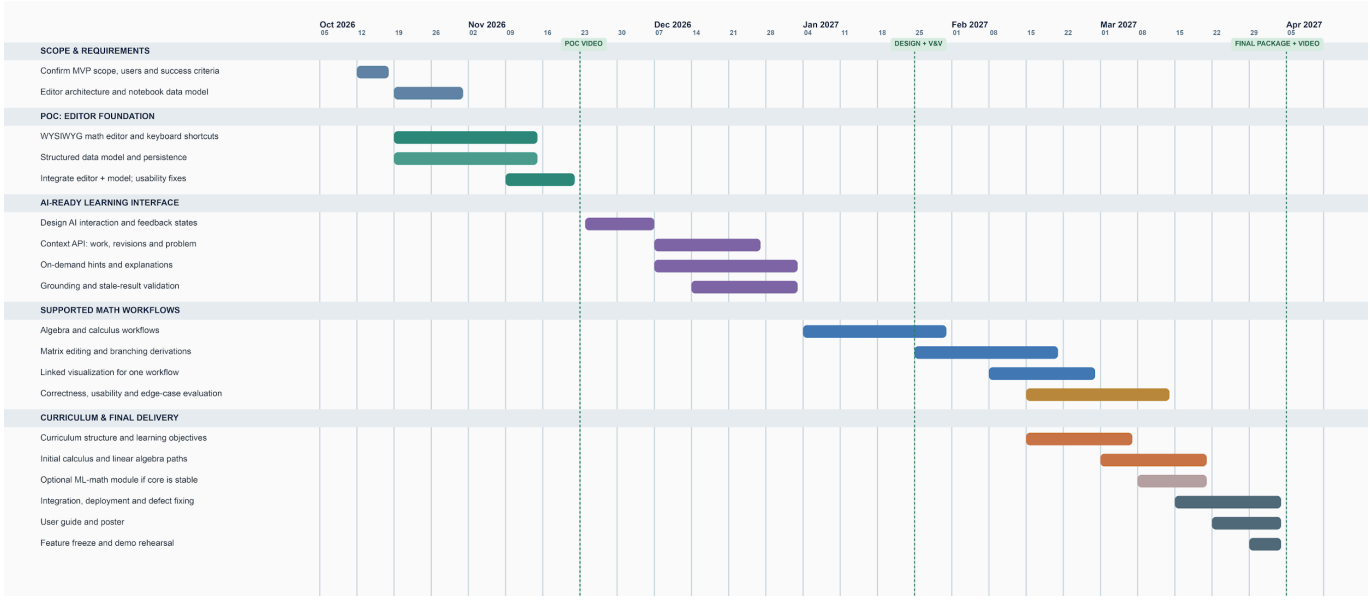



Figure 1. Folio development schedule, October 2026–April 2027.

Appendix A: Generative AI Usage Disclosure

Generative AI tools, including ChatGPT, were used to brainstorm, refine technical explanations, organize requirements, and improve clarity. The team reviewed and revised all generated material. All requirements, architectural decisions, and project plans are the team’s responsibility.